

Hệ Thống Kiểm Duyệt Phát Ngôn Thù Ghét Tiếng Việt (ViHSD)

Dựa trên nền tảng MLOps & Agentic AI dành cho môi trường doanh nghiệp

Nhóm sinh viên thực hiện:
23120345 - Nguyễn Trịnh Quang
23120364 - Phạm Minh Thông
23120369 - Nguyễn Lê Tân Tiến
23120392 - Đào Quốc Tuấn

Mục tiêu & Ứng dụng



Mục tiêu

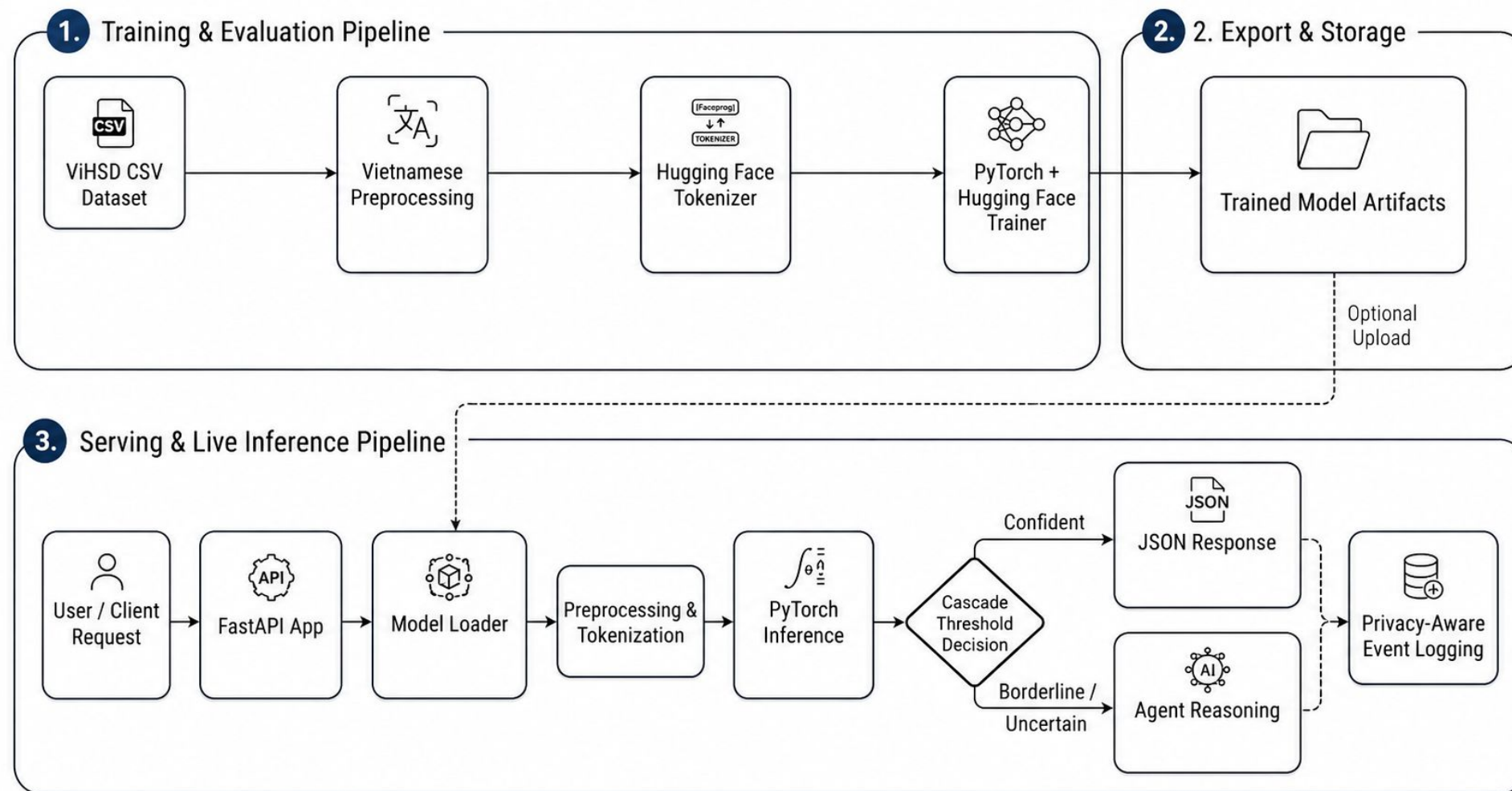
- Phát hiện ngôn từ thù ghét trong môi trường doanh nghiệp Việt Nam.
- Tự động hóa từ tiền xử lý đến phục vụ mô hình thực tế.
- Tối ưu hóa khả năng nhận diện các biến thể ngôn ngữ phức tạp.



Ứng dụng

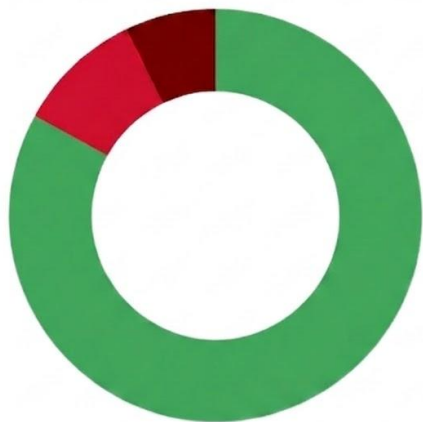
- Hệ thống phân loại văn bản nội bộ cho doanh nghiệp có dùng mạng xã hội.
- Bộ lọc cho Chatbot
- Kiểm duyệt nội dung tự động trên các nền tảng mạng xã hội

Pipeline



EDA

Mất cân bằng nhãn



- CLEAN: 5,518 mẫu (82.98%)
- HATE: 688 mẫu (10.35%)
- OFFENSIVE: 444 mẫu (6.67%)

Đặc trưng văn bản của dataset

- Độ dài chủ yếu: 10 - 45 từ.
- Nhiều yếu tố gây nhiễu: kí tự đặc biệt, emoji, teencode, từ viết sai chính tả hoặc từ lóng,...

Giải pháp được sử dụng:

- Độ dài tối ưu cho config training: 128 tokens
- Sinh thêm dữ liệu cho class thiểu số bằng kỹ thuật random deletion, random swap.
- Áp dụng ngưỡng xác suất phi đối xứng cho class OFFENSIVE, HATE.

Preprocessing



Lọc nhiễu

Xóa URLs, HTML,
@mentions & lọc văn bản
ngắn < 3 ký tự.



Chuẩn hóa

Mapping Unicode NFKC, xử
lý ký tự lặp (đẹpppp →
đẹp).



Ngữ nghĩa

Khôi phục teencode (ko →
không), giữ sắc thái nhấn
mạnh.



Pipeline

Chuyển đổi từ văn bản thô
sang định dạng Vector
sạch.

Baseline Model



TF-IDF + Logistic Regression

Sử dụng n-grams (Word: 1-3, Char: 2-5) để trích xuất đặc trưng tĩnh.

- **Ưu điểm:** Huấn luyện nhanh, không cần GPU.
- **Accuracy:** 80.53%
- **Macro F1:** 62.50%



Hạn chế

Không nắm bắt được ngữ cảnh toàn cục và đánh giá sai phủ định phức tạp.

Ví dụ: "Không ghét chút nào" → Có thể bị nhận diện sai nhãn Hate do chứa từ khóa "ghét" và không có ngữ cảnh cụ thể.

Backbone Model

Mô hình

xlm-roberta-base: 278M tham số, 100 ngôn ngữ, có thiên hướng nhận diện độc hại, xử lý tổ teencode.

PhoBERT Base: 135M tham số, tối ưu sâu cho ngữ pháp tiếng Việt đặc thù.

Kiến trúc & Quản lý

Phân loại: AutoModelForSequenceClassification tiêu chuẩn của HuggingFace.

Phân phối: Đóng gói và đẩy trực tiếp lên HuggingFace Hub Registry.

Mô hình	Accuracy	Macro F1	CLEAN F1	OFFENSIVE F1	HATE F1
Baseline (TF-IDF + LR)	80.53%	62.50%	89.65%	41.23%	56.61%
PhoBERT Base (HF)	85.11%	63.36%	92.41%	41.06%	56.60%
XLM-R Base (Đề xuất)	86.74%	64.61%	93.62%	41.13%	59.09%

Cascade Thresholds Decision

Cascade Thresholds

Khắc phục Class Imbalance: Cascade Threshold đặt ngưỡng riêng cho từng lớp nên lớp thiểu số vẫn được chọn khi đạt đủ mức tin cậy.

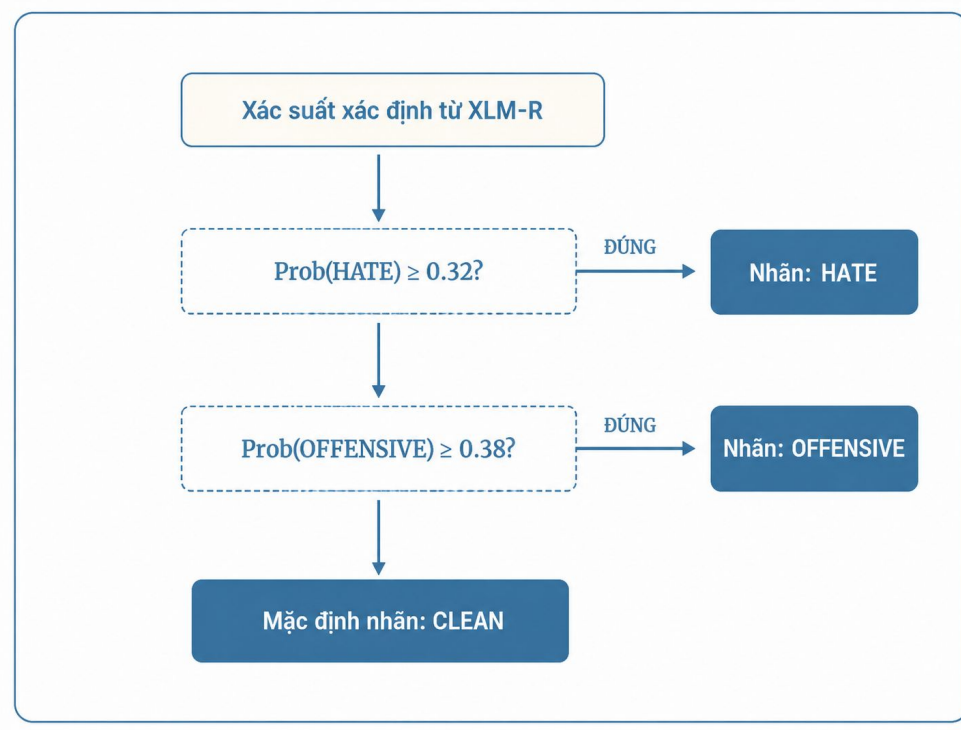
Asymmetric Classification kiểm duyệt tuần tự theo thứ tự ưu tiên ưu việt.

Cấu hình ngưỡng thực tế: Thiết lập ngưỡng nhận diện phân tầng tối ưu: HATE: 0.32 | OFFENSIVE: 0.38 |

→ CLEAN: 0.50.

Kết quả: Giữ nguyên độ chính xác tổng thể, đồng thời tăng vọt tỷ lệ bắt trúng nhãn HATE lên 58.57%.

Luồng Logic Nhận Diện



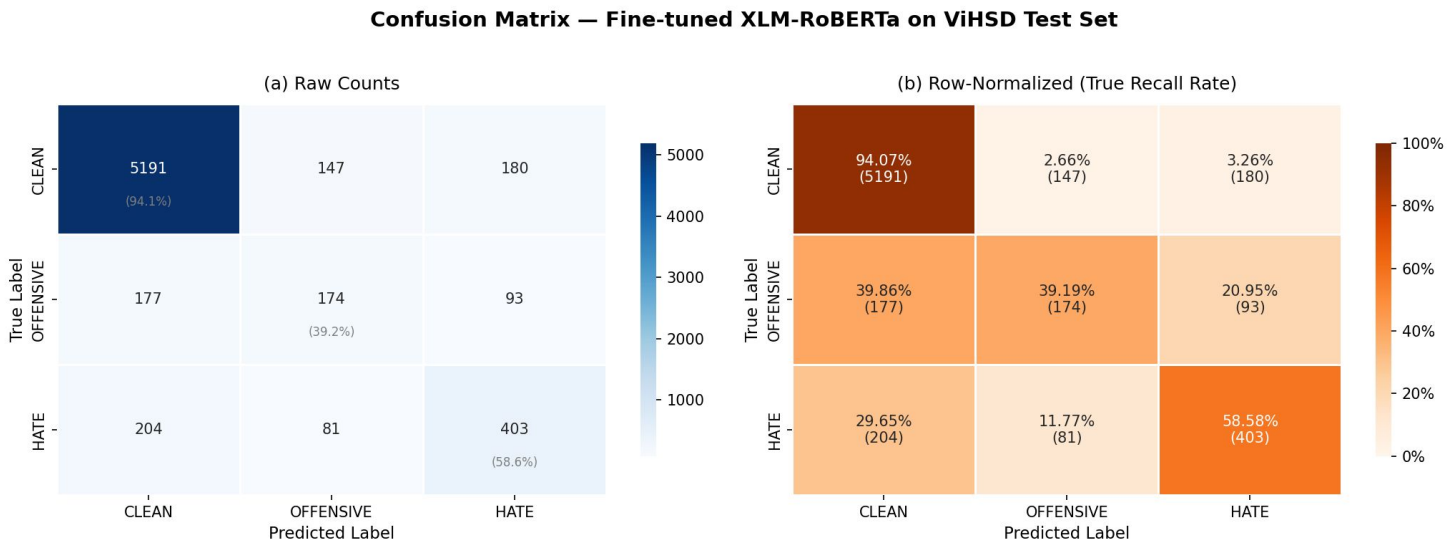
Error Analysis

Tỷ lệ phân phối nhầm lẫn chuẩn hóa

- CLEAN (5,518 mẫu):** Dự chính xác **94.07%** | Nhầm Offensive 2.66% | Nhầm Hate 3.26%.
- OFFENSIVE (444 mẫu):** Dự chính xác **39.19%** | Nhầm Clean **39.86%** | Nhầm Hate 20.95%.
- HATE (688 mẫu):** Dự chính xác **58.58%** | Nhầm Clean **29.65%** | Nhầm Offensive 11.77%.

Các ca phân loại sai điển hình

- Lỗi ẩn dụ/mĩa mai:** "Bạn thông minh thế này thì nên về quê chăn vịt đi" → Dự đoán CLEAN (Thực tế: OFFENSIVE).
- Lỗi từ khóa nhạy cảm:** "Con chó nhà em rất ngoan và đáng yêu" → Dự đoán HATE/OFFENSIVE do từ "chó".



Local Core & Remote GPU Worker

Nguyên tắc thiết kế hệ thống

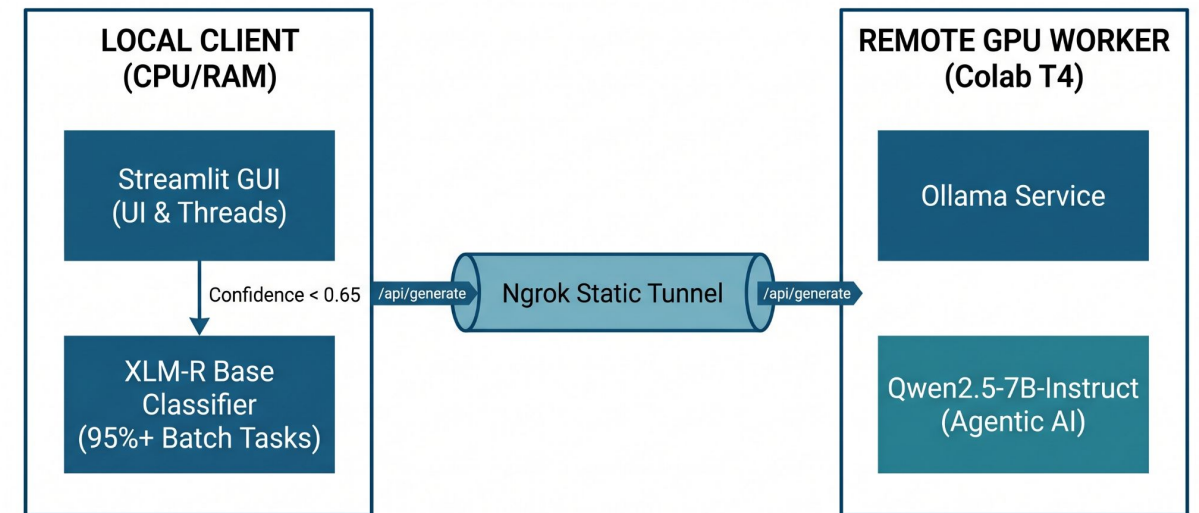
Tách biệt hoàn toàn luồng xử lý định tuyến cục bộ (CPU/RAM) và luồng suy luận ngôn ngữ lớn (GPU VRAM) để tối ưu hóa chi phí phần cứng.

LOCAL CLIENT

- Trung tâm điều phối, chạy Streamlit GUI và bộ nhớ trạng thái đa luồng.
- Vận hành mô hình **XLM-R Base** đánh giá hàng loạt & định tuyến dữ liệu.

REMOTE WORKER

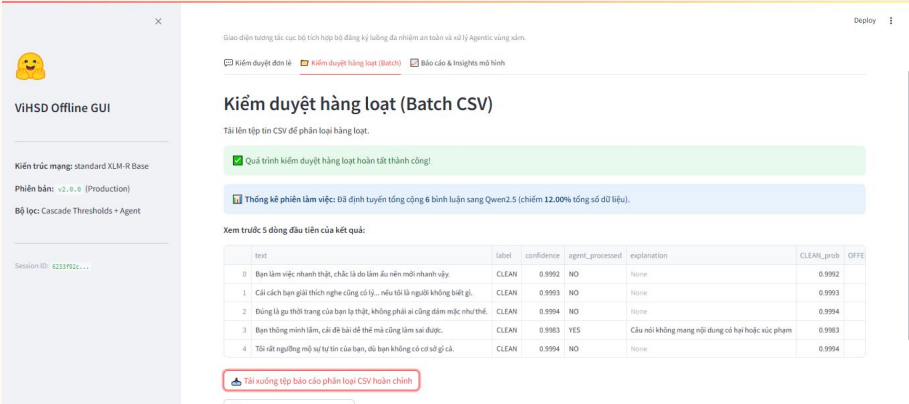
- **Ollama** trên GPU T4 (16GB VRAM) nạp Qwen2.5-7B-Instruct (Int4).
- Kết nối qua **Ngrok Static Domain** duy trì endpoint `/api/generate` ổn định.



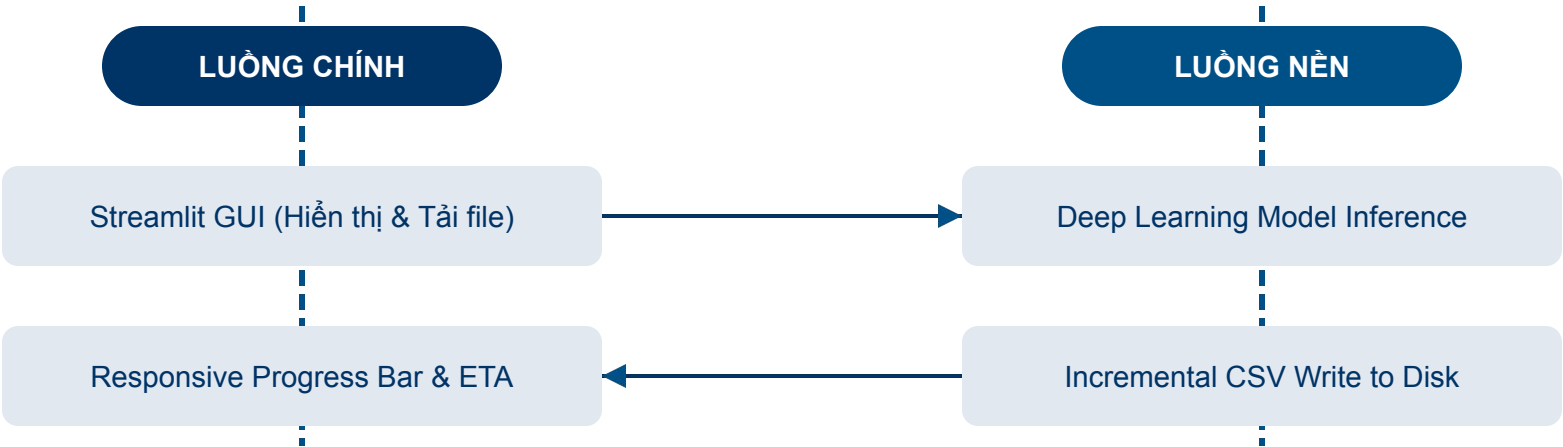
Local GUI Client & Batch Processing



- **Thách thức:** Đọc file CSV lớn và chạy mô hình deep learning trực tiếp trên luồng hiển thị sẽ làm giao diện bị đơ/đóng băng (UI Freezing).
- **Giải pháp Đa luồng:** Đẩy quá trình phân loại xuống luồng nền (threading.Thread) giúp giao diện luôn mượt mà.
- **Xử lý cuộn chiếu:** Chia nhỏ file CSV (10-20 dòng) ghi nối ổ đĩa, bảo vệ RAM.



Cơ Chế Song Song (Concurrency)

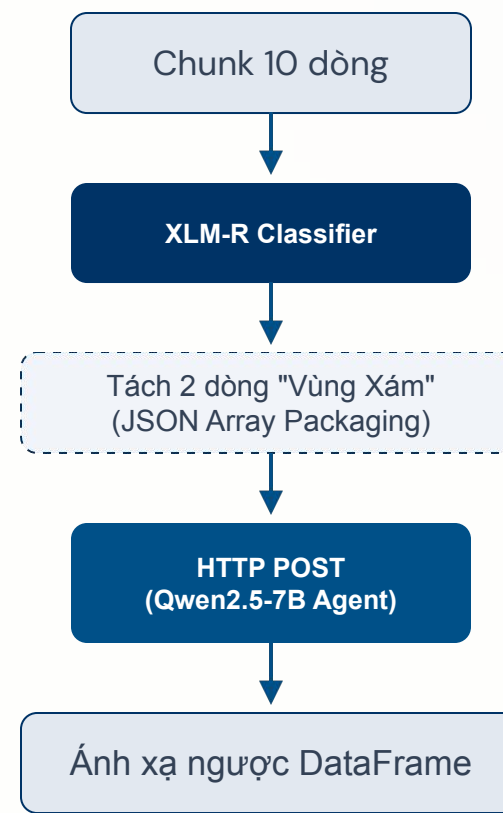


Agentic AI Layer

Phân định ranh giới thực thi

- **Bộ định tuyến chuyên biệt (XLM-R):** Đảm nhận đánh giá ~85% lưu lượng văn bản rõ nghĩa với chi phí phần cứng bằng không.
- **Gom cụm vùng xám:** Khi xử lý lỗi, hệ thống trích xuất M mẫu có độ tự tin mờ (**confidence < 0.65**), gán định danh (ID) và đóng gói thành một **JSON Array**.
- **Tối ưu hóa độ trễ tuần tự:** Gửi mảng JSON qua Endpoint trong một **HTTP Request duy nhất** thay vì gọi API M lần.

Sơ đồ dòng chảy dữ liệu



Containerized Serving API



Đóng gói & Cấu hình Container

- Sử dụng Containerization tạo môi trường cô lập, nhất quán 100%, tránh xung đột thư viện giữa Dev và Ops.
- **Dockerfile:** Dùng python:3.10-slim tối ưu kích thước.
- **docker-compose.yml:** Map cổng container ra host tại 8000.

FastAPI Endpoints Tiêu chuẩn

- **GET /health:** Kiểm tra sức khỏe hệ thống.
- **GET /ready:** Xác nhận worker đã nạp xong mô hình.
- **POST /predict:** Phân loại chuỗi văn bản + **langdetect** tự động.
- **POST /predict-batch:** Xử lý mảng hiệu năng cao.

The screenshot shows a web browser window with the address bar at `localhost:8000/docs#/Inference/predict_predict_post`. Below the browser, there is a REST client interface with a blue "Execute" button and a "Clear" button. The "Responses" section is expanded, showing the following details:

- Curl:**

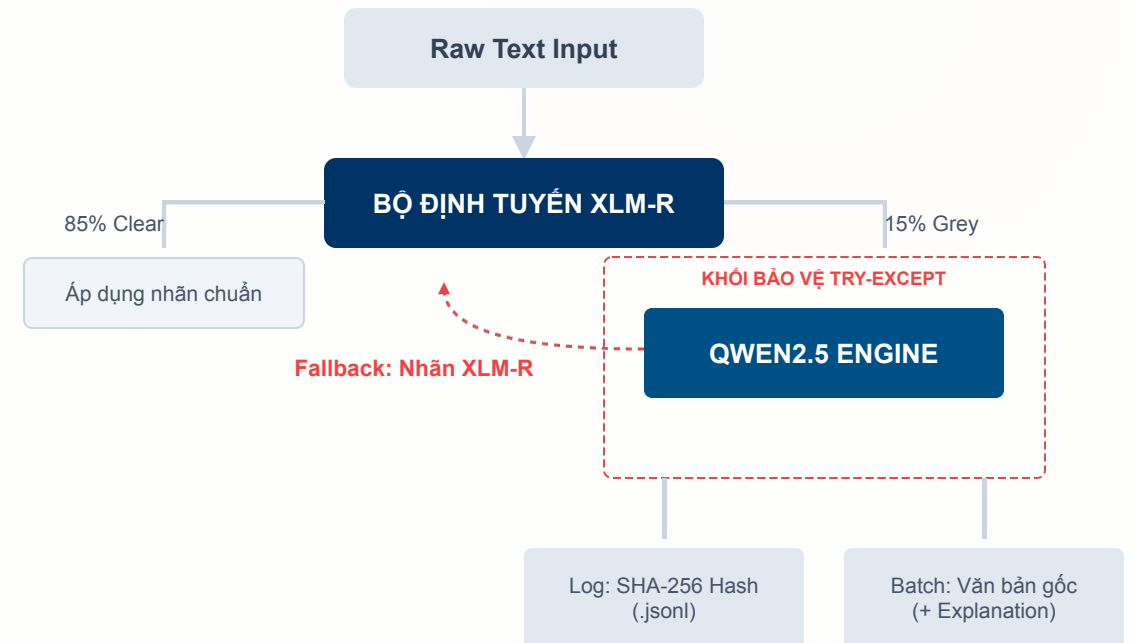
```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "text": "não máy bị úng à",
    "language": "vietnamese"
  }'
```
- Request URL:** `http://localhost:8000/predict`
- Server response:**

Code	Details
200	Response body <pre>{ "text": "não máy bị úng à", "label": "HATE", "confidence": 0.9971, "probabilities": { "CLEAN": 0.000, "OFFENSIVE": 0.0009, "HATE": 0.9971 }, "model_version": "v2.0.0", "language": "vietnamese" }</pre> Response headers <pre>access-control-allow-credentials: true</pre>

Ethics, Privacy & Safety Risks

- **Tuân thủ Nghị định 13/2023/NĐ-CP:** XLM-R xử lý biên 85%, chỉ 15% dữ liệu "vùng xám" đi vào Qwen2.5 nội bộ, triệt tiêu rủi ro rò rỉ dữ liệu qua Cloud API và tối ưu chi phí hạ tầng.
- **Bảo mật nhật ký SHA-256:** Nội dung thô được băm một chiều khi lưu *.jsonl*. Chỉ tệp Batch CSV cho kiểm duyệt viên mới giữ văn bản gốc kèm cột giải thích.
- **Cơ chế Fallback an toàn:** Khởi lệnh **try-except** tự động bắt lỗi mạng/JSON. Nếu LLM từ chối, hệ thống dùng nhãn thô của XLM-R làm kết quả phòng vệ.

Chiến lược Phân tầng & Hạ cấp (Fallback)



Key Takeaways & Future Work

Kết luận thực tế

- **Kiến trúc Hybrid tối ưu chi phí:** Mô hình XLM-R Base giải quyết >85% tác vụ rõ nghĩa tốc độ cao, chỉ 15% các câu mờ sang Qwen2.5 nội bộ bảo mật, giảm chi phí Cloud API về không.
- **MLOps vững chắc, kháng lỗi sâu:** Hệ thống xử lý hàng loạt mượt mà nhờ chunk_size=10, chống deadlock luồng đĩa cứng và phòng vệ tự động bằng cách trích xuất nhãn mặc định khi LLM từ chối.

Định hướng phát triển

- **Tăng cường cân bằng dữ liệu:** Áp dụng Data Augmentation cải thiện F1-Score lớp OFFENSIVE, HATE (hiện đạt 41.13% do mất cân bằng nghiêm trọng).
- **Tự chủ hạ tầng phần cứng:** Dịch chuyển Ollama từ Google Colab sang cụm GPU On-premise cố định, loại bỏ phụ thuộc Ngrok Free.

Lộ trình phát triển



GIẢI PHÁP HIỆN TẠI
Hạ tầng MLOps Hybrid Core +
Agent cục bộ vững chắc

GIẢI ĐOẠN TIẾP THEO
Mở rộng phân phối dữ liệu &
Hạ tầng On-premise độc lập

CẢM ƠN THẦY VÀ CÁC BẠN!

Vì đã lắng nghe và đồng hành cùng buổi thuyết trình

